A Foundation for Self-Optimizing Retrieval-Augmented Generation

Architecture, optimization theory, compositional IR/DSL, plugin model, and engineering workflow

Code and design analysis of CoinVault/GEC, RAG-TTC, ragkit, and ragopt

2026-08-12

Contents

1 Executive summary

The two implementations are not yet self-optimizing RAG systems in the strict sense. They are strong **candidate-proof and evidence-custody systems** around manually authored RAG changes.

CoinVault/GEC has the stronger proof loop. It freezes a suite and product identity, executes incumbent/challenger pairs, enforces shared budgets, records rich traces, verifies that a treatment was actually exercised, resumes interrupted work, classifies failures, and emits a terminal gate and promotion plan. Its weakness is structural: most product semantics and every supported mutation mechanism are encoded in a large application command and a family of adjacent product-specific files. Adding a new optimization dimension generally means adding another branch, trace projection, validator, configuration shape, and source lock.

RAG-TTC has a thinner `ragopt` adapter and a well-developed design for a GEPA-inspired reflection workflow. It also contains separate answer-quality and chunking experiment runners that already cover more of the RAG surface. Its implemented `ragopt` loop is nevertheless a fixed experiment: one hand-authored asset mutation, hard-coded runtime YAML, a fixed provider profile, a fixed judge, and two projected answer metrics. The broader dimensions exist, but across separate commands rather than a common search space and execution model.

`ragkit` is a credible data-plane foundation. It provides narrow interfaces, strong lineage types, deterministic ordering, immutable content-addressed index bundles, observable answer stages, bounded execution, semantic cache identities, retries, admission controls, and replay-oriented flow steps. `ragopt` is a credible evidence-plane foundation. Its pinned versions provide immutable snapshots, exactly-one-mutation candidates, paired cells, run manifests, append-only artifacts, resumability, strict comparison and gates, tamper-evident cell chaining in v0.0.1, and structurally blinded human review. The missing layer is a **declarative, versioned, optimizable program representation** plus search engines that can propose and schedule candidates.

The recommended direction is therefore not to replace `ragkit` or discard `ragopt`. It is to establish a six-part architecture:

1. **RAG mechanism/data plane (`ragkit`)** - typed components for ingestion, representations, indexing, retrieval, reranking, context construction, generation, tools, and validation.
2. **Plan and compiler plane** - a serializable typed graph IR, component registry, schema validation, capability/effect checking, canonical identities, dependency analysis, and cost preflight.
3. **Experiment control plane (`ragopt`)** - studies, campaigns, trials, cells, attempts, immutable artifacts, resumability, budgets, scheduling, and external executor interfaces.

4. **Evaluation and analysis plane** - generic traces, deterministic validators, retrieval/answer/tool/system metrics, LLM and human judges, statistical comparison, diagnostics, and a rebuildable warehouse.
5. **Search and learning plane** - manual hypotheses, random/Sobol screening, TPE/SMAC, Hyperband/BOHB, Pareto evolutionary or Bayesian methods, and GEPA-style textual proposers.
6. **Promotion and governance plane** - hard constraints, non-inferiority gates, Pareto archives, audit sets, blinded review, canaries, activation plans, and rollback.

The key abstraction should be:

```
StudySpec + registries
        |
        v
strict compiler --> canonical Plan + Lockfile + SearchSpace
        |                          |
        |                          +--> typed Patch candidates
        v
content-addressed execution DAG --> traces/artifacts/metrics
        |
        v
optimizer observes results --> proposes more Patches
        |
        v
validation/audit/gates --> reviewed promotion plan
```

The DSL should remain declarative and non-Turing-complete. YAML, JSON, or CUE can be the authoring syntax; CEL-like expressions can define activation conditions, constraints, and gates; scripts should be explicit, digest-pinned escape hatches rather than hidden semantics. Both the textual DSL and a Go builder should compile to the same canonical IR. The IR, not YAML and not application branches, becomes the execution contract.

A central design rule is: **make every dimension representable, but do not mutate every dimension simultaneously by default**. Exactly-one-target experiments remain valuable for causal attribution and should survive as a `proof` protocol. Broader search should use block-coordinate stages, controlled interaction rounds, and multi-fidelity evaluation. This avoids both the current rigidity and an unidentifiable combinatorial search.

2 1. Scope, method, and limitations

2.1 1.1 Material analyzed

The analysis covers the two supplied archives and the exact public `ragopt` revisions referenced by them:

- CoinVault/GEC application source from `gec-rag-opt.zip`.
- The extracted `ragkit` source included with that archive.
- RAG-TTC application source from `rag-ttc(5).zip`.
- `ragopt` commit `4d410c57e242`, referenced by CoinVault as a pseudo-version.
- `ragopt` tag `v0.0.1`, commit `0e9c584fee2d`, referenced by RAG-TTC.
- Local RAG-TTC design documents for the GEPA-inspired optimizer, evaluation roadmap, and post-cutover package ownership.

The repositories are substantial rather than toy sketches:

Repository	Production Go files	Test Go files	Approx. production LOC	Approx. test LOC
CoinVault/GEC	162	95	32,719	14,813
ragkit	139	74	16,777	11,830

Repository	Production Go files	Test Go files	Approx. production LOC	Approx. test LOC
RAG-TTC	231	129	44,644	19,572

The focused CoinVault RAG optimization command is about 1,636 lines before its adjacent case, contract, gate, reranker, suite-lock, trace, and treatment files. The RAG-TTC `ragopt` adapter is about 456 lines, while its broader answer-quality runner is about 1,165 lines.

2.2 1.2 Validation limitation

This is a static source, configuration, and design analysis. The supplied modules require Go 1.26.5 and newer `go.mod` syntax, while the available local toolchain is Go 1.23.2. Toolchain download was blocked by the execution environment's network policy. Consequently, repository test suites could not be executed here. The report distinguishes observed source behavior from proposed design and does not claim runtime verification.

2.3 1.3 Terminology

The report uses the following terms consistently:

- **System**: one complete RAG program/configuration, including offline index construction and online query execution.
- **Plan**: a canonical, compiled, executable representation of that system.
- **Patch**: a typed change against a parent plan.
- **Study**: datasets, objectives, constraints, search space, fidelities, budgets, and promotion policy.
- **Campaign**: one optimization history under one immutable study identity.
- **Trial**: evaluation of one plan/patch at one declared fidelity.
- **Cell**: one case, repeat, arm, and environment combination.
- **Attempt**: one execution attempt for a cell.
- **Metric**: a versioned observation with direction, unit, level, aggregation, and missingness semantics.
- **Gate**: a deterministic decision policy over evidence; it is not itself an optimizer.

3 2. What exists today

3.1 2.1 ragkit: a strong mechanism library, not yet an optimizable program model

`ragkit` follows a contracts-first structure. The root `rag` package defines source-lineage types and narrow interfaces for chunking, generation, embedding, search, indexing, and reranking. Implementations live in replaceable packages. Important properties include:

- `Chunk` is an exact byte range of an immutable `Document`.
- `Representation` is explicitly retrieval material, while the source `Chunk` remains evidence.
- Provider-facing interfaces preserve usage even when a call returns an error.
- Retrieval order is complete and deterministic.
- Prompt digests and model identities participate in representations and experiment identity.
- Persistent index bundles bind corpus, chunk, representation, lexical, vector, and content identities.
- The answer service retains channel rankings, variants, evidence, context admission, provider requests, raw generation, parsed answers, contract results, and stage observations.
- The generic `flow.Step[I,O]` separates semantic cache identity from workers, retries, and other execution policy.

This is the right direction. In particular, the distinction in `flow.Step` between **what determines output** and **how work is executed** should become a framework-wide identity rule. It allows safe replay, common-subexpression reuse, and independent tuning of performance policy.

The limitation is that composition is currently a Go value graph. A `Step` carries functions such as `Identity.Key`, `Do`, and `OnResult`; the answer service selects behavior through a closed strategy enum and direct fields. These are excellent runtime APIs, but an optimizer cannot reliably:

- serialize and diff the entire pipeline;
- enumerate legal mutation sites;
- inspect component schemas and capabilities;
- replace a node without application code;
- reason about build/query dependencies;
- compute a canonical plan identity independent of a process;
- discover which trace assertions prove that a mutation was exercised.

`ragkit` therefore needs descriptors and adapters around its components, not a rewrite of its domain types.

3.2 2.2 ragopt: a strong evidence protocol, not yet an optimizer

At the versions used by the applications, `ragopt` defines a strict experiment protocol:

- A snapshot records locked and mutable assets plus dimensions.
- A candidate records parent/child snapshots, proposer identity, hypothesis, expected improvement, regression risks, and selected diagnostic cases.
- Loading independently verifies that exactly one mutable asset changed and that locked assets and dimensions did not change.
- A suite is an ordered set of stable, product-defined cases with opaque JSON input.
- Product code implements an `Arm` that receives a bounded candidate view and returns a generic outcome plus a native artifact.
- The runner executes incumbent/candidate pairs per case and repeat, persists cells, validates native artifact boundaries, and resumes only under exact copied-input and configuration identity.
- Comparison computes paired metric and cost deltas by case and group.
- Gate policies enforce completion, contract validity, failure-rate and metric floors, one target improvement, regression bounds, and lexicographic cost tie-breakers.
- The run store is append-only while active and immutable after a terminal state.

RAG-TTC's `v0.0.1` adds meaningful custody improvements over the CoinVault-pinned revision: cells carry a previous digest and digest, the journal is chained, arm execution is guarded against mutation of the run trust roots, and a generic blinded human-review protocol is available. Those should be preserved.

The package nevertheless does not perform optimization. It has no first-class abstraction for:

- a search space;
- a proposal algorithm;
- a canonical RAG graph;
- structural or multi-target patches;
- candidate ancestry beyond one parent/child bundle;
- a campaign archive or Pareto frontier;
- adaptive fidelity allocation;
- statistical confidence or sequential selection;
- optimizer state and ask/tell APIs;
- dependency-aware index build reuse;
- trace-informed reflection.

The best description is: **`ragopt` is a reproducible evidence and promotion substrate on which an optimizer can be built**.

3.3 2.3 CoinVault/GEC: rigorous proof loop with application-specific orchestration

The CoinVault command implements a complete proof cycle around native product execution:

1. Decode and reject budget changes from locked constants.
2. Resolve a frozen feedback or validation suite and repeat count.
3. Load the candidate bundle and verify suite cases.
4. Build a judge runtime and verify models, profiles, bundle, source locks, and candidate dimensions.
5. Run a preflight-only mode without provider calls.
6. Seed shared answer, embedding, token, and judge budgets when resuming.
7. Construct incumbent and challenger arms for `ragopt`.
8. Execute or resume the paired matrix.
9. Persist a terminal gate, review artifact, and promotion plan.

Its cell executor is unusually careful. It loads a locked treatment contract, interprets the mutable asset, configures the native runner, collects provider/tool/evidence traces, applies a timeout boundary, classifies ownership of failures, builds deterministic answer/grounding contracts, invokes a judge when appropriate, verifies treatment activation, and writes a native artifact.

The treatment model covers at least these dimensions:

- result defaults and forced result limits;
- comparison decomposition;
- comparison intent;
- grounding, routing, and policy prompt suffixes;
- tool descriptions;
- reranker configuration.

This demonstrates several mechanisms worth extracting as generic concepts:

- **Treatment declaration**: a patch says what is supposed to change.
- **Invariant declaration**: semantic identities that must remain fixed are explicit.
- **Activation probe**: a trace proves the candidate configuration reached the runtime.
- **Exercise probe**: a case proves the changed behavior was relevant and used.
- **Failure ownership**: adapter timeout, provider failure, tool failure, projection failure, contract failure, and judge failure are not collapsed.
- **Budget custody**: uncertain timeout usage closes budgets conservatively and resume reconstructs spend from artifacts.

The core problem is that these concepts are embodied as CoinVault structs and branches. The cell executor contains a mechanism switch that opens a different asset and configures a different native runtime path for each mutation type. The trace collector understands CoinVault event names and `knowledge_search` payloads. The command name and description still contain the original default-results proof-cycle wording even while the same code handles many mechanisms. Each new dimension expands the adapter rather than registering a component, schema, patch target, and trace assertion.

A concrete configuration-drift example reinforces the need for compilation from one source of truth. The command locks:

```
answer calls       216
embedding calls    192
judge calls         72
answer tokens  1,000,000
```

The copied and hashed `default-results-8-v7/shared/runtime-contract.yaml` declares:

```
answer calls       216
embedding calls    192
judge calls         24
provider tokens 2,000,000
```

The runtime contract is part of snapshot identity, but the command enforces its code constants. A candidate can therefore carry a locked document that does not describe the executed ceilings. A compiler-generated lockfile should make this class of disagreement impossible.

3.4 2.4 RAG-TTC: thinner adapter, stronger forward design, fragmented dimensions

The RAG-TTC `ragopt` command is closer to the intended product adapter boundary. It loads a candidate/suite, verifies identities, constructs two arms, materializes a few assets, executes the native chat runtime, judges the answer, and projects faithfulness, relevance, contract validity, abstention, calls, tokens, and duration into a generic outcome.

That thinness is positive. The adapter is not, however, generic configuration:

- required asset names are hard-coded;
- provider and model choices are fixed;
- tool-loop and retrieval configuration is emitted through one large `fmt.Sprintf` YAML literal;
- the implemented candidate mutates one search description;
- there is no generic treatment activation assertion;
- the projected metric set is narrow;
- the command does not itself close a separate validation/audit protocol.

RAG-TTC's broader experiment tree contains chunk-comparison and answer-quality runners that exercise chunkers, representations, retrieval variants, generation, judging, human review, budgets, and artifacts. This is evidence that the required dimensions already exist, but as separate orchestration programs rather than one compositional study model.

The local GEPA design is strong and should be retained. It correctly proposes:

- append-only JSONL as authority and disposable SQLite as an analysis index;
- a feedback split visible to the reflector, validation for decisions, and held-out audit before promotion;
- rich trajectory and textual feedback rather than scalar reward alone;
- exactly one changed textual component for early causal clarity;
- complete replacement files rather than free-form model patches;
- hard gates, repeated validation, and human review;
- strict exclusion of judge prompts and evaluation answers from the mutation surface;
- candidate lineage and eventual Pareto-style search.

The missing step is to connect that workflow to a generic plan/search kernel rather than implement a second product-specific reflection command.

4 3. Comparative assessment

Capability	CoinVault/GEC	RAG-TTC	ragkit/ragopt foundation
Reproducible inputs	Very strong source, suite, model, bundle, policy, and asset locks	Strong asset/profile/index locks	Strong snapshots, manifests, copied inputs, content identities

Capability	CoinVault/GEC	RAG-TTC	ragkit/ragopt foundation
Resume and evidence custody	Strong, including shared budget reconstruction	Strong through ragopt, simpler product accounting	Strong; v0.0.1 adds cell chain and evidence guard
Product trace richness	Very high but application-specific	Good session/native artifacts	ragkit has useful stage observations; no unified cross-product trace schema
Treatment activation verification	Excellent	Minimal	Not first-class in ragopt
Candidate generation	Manual	Manual; reflective design only	No search/proposer abstraction
Search across indexing and query dimensions	Separate code paths and hand-authored candidates	Separate experiment commands	No serializable joint space or dependency planner
Multi-objective handling	Hard gates, one target, regressions, cost tie-breakers	Same underlying model	Lexicographic policy, not a Pareto campaign archive
Statistical decision quality	Paired repeats, but limited uncertainty treatment	Paired repeats, narrow metrics	No confidence intervals or multiple-comparison controls
Compositionality	Native code and mechanism switch	Native code and hard-coded YAML	Typed Go interfaces and `Step` composition; not optimizer-inspectable
DSL / IR	Candidate YAML and product contracts, no system IR	Candidate YAML and runtime config, no system IR	No canonical RAG plan IR
Plugin model	Product code compiled into app	Product code compiled into app	Narrow interfaces, no descrip-tors/registry/protocol
Promotion governance	Strong non-applying plan	Designed and partly available through ragopt review	Strong base, needs audit/canary/rollback integration

4.1 3.1 Preserve

The new foundation should preserve, largely unchanged in spirit:

- ragkit source lineage and evidence/material distinction;
- deterministic ordering and content-addressed index bundles;
- semantic cache identity separated from execution policy;
- ragopt immutable copied inputs, run lifecycle, native artifact custody, comparison, gates, chained cells, and blinded review;
- CoinVault's treatment activation, failure ownership, and conservative budget accounting;
- RAG-TTC's feedback/validation/audit separation and JSONL-to-SQL analysis workflow;
- product ownership of source extraction, safety semantics, serving integration, and deployment activation;
- external ownership of cron/distributed job execution, as already recommended in the ragopt production-refresh design.

4.2 3.2 Replace or generalize

The foundation should replace:

- large mechanism enums and switches with component descriptors and patch targets;
- copied arbitrary file trees as the only candidate representation with typed plan patches;
- hard-coded runtime YAML and duplicated constants with compiler-generated plans and lockfiles;
- product-specific trace collectors with a generic trace envelope plus schema-qualified domain payloads;
- one target metric plus tie-breakers as the only search view with constraints plus a Pareto archive;
- fixed incumbent-first scheduling with persisted randomized or counterbalanced paired schedules;
- separate experiment runners for chunking, retrieval, prompts, and tools with one study model and reusable graph subplans;
- manual proposal as the only path with pluggable ask/tell optimizers and GEPA-style proposers.

5 4. Formal problem statement

5.1 4.1 A RAG system is a typed attributed program graph

Represent a system as:

$$x = (G, c, \theta, p)$$

where:

- $G = (V, E)$ is a typed directed graph;
- c_v selects a component implementation for node v;
- θ_v is the component configuration;
- p is execution policy such as workers, retries, placement, batching, and budgets.

A complete graph contains two related subgraphs:

```
OFFLINE / REFRESH GRAPH
source -> parse -> normalize -> chunk -> represent -> embed -> index -> publish

ONLINE / QUERY GRAPH
question -> rewrite/route -> retrieve -> fuse -> rerank -> hydrate
         -> context select -> generate/tool loop -> validate -> answer
```

A change to an offline ancestor invalidates descendants and may require a new index artifact. A query-only change should reuse the immutable index. This makes evaluation scheduling a content-addressed DAG and common-subexpression-elimination problem, not merely a Cartesian loop over configurations.

5.2 4.2 Outcomes are stochastic, grouped, and expensive

For case z_i, environment e, repeat/seed r, and system x:

$$Y_{i,r}(x) = F(x, z_i, e, r)$$

The observation includes answer and retrieval quality, failures, calls, tokens, latency, resource use, artifacts, and trace. Metrics are functions $\phi_j(Y)$, and the desired objective vector is:

$$\mu(x) = (\mathbb{E}[\phi_1(Y)], \dots, \mathbb{E}[\phi_m(Y)])$$

There is no honest universal scalar objective. A useful vector spans at least:

- correctness and task utility;
- faithfulness and citation support;
- retrieval coverage and ranking quality;
- completeness and instruction satisfaction;
- calibrated abstention;
- safety and authorization;
- provider/tool failure rate;
- latency percentiles;
- calls, tokens, dollar cost, memory, storage, and index build cost;
- robustness across case groups, time, models, and environment variants.

Hard constraints are represented separately:

$$g_k(x, Y) \leq 0, \quad k = 1, \dots, K$$

Examples are zero critical safety regressions, contract validity floors, maximum failure probability, memory ceilings, or authorization invariants.

5.3 4.3 Indexing cost must be amortized explicitly

Joint indexing/query optimization is distorted if only per-query cost is measured. For an expected serving horizon H:

$$C_H(x) = C_{build}(x) + H\,\mathbb{E}[C_{query}(x)] + C_{storage}(x) + C_{refresh}(x)$$

The study should either state H or report build and query costs separately. A large index can be optimal for a high-volume service and irrational for a low-volume or frequently refreshed corpus. Refresh frequency, incremental reuse, and deployment footprint are therefore study inputs, not hidden assumptions.

5.4 4.4 Paired comparison is necessary but not sufficient

For incumbent x_0 and candidate x_1, use paired differences:

$$d_{i,r} = m(Y_{i,r}(x_1)) - m(Y_{i,r}(x_0))$$

Pair the same cases, environment, and, where providers support it, random seeds or common random numbers. Persist the schedule before execution. Randomize or counterbalance arm order inside blocks; the current incumbent-then-challenger order can confound provider drift, warming, rate limits, or shared caches.

Promotion should require more than a positive sample mean. A practical policy is:

1. all hard constraints pass;
2. a lower confidence bound for the primary paired improvement exceeds a declared margin;
3. upper confidence bounds for regressions remain inside non-inferiority margins;
4. the candidate is non-dominated or improves the chosen operating region of the Pareto frontier;
5. audit and human-review requirements pass.

Use case-level or hierarchical bootstrap intervals, with repeats nested inside cases. Missing results are typed failures or missing metrics, never silently converted to zero. When many candidates are adaptively tested, protect the audit set and use sequential or multiple-comparison controls rather than selecting the largest noisy validation result.

5.5 4.5 Represent all dimensions; search in controlled blocks

The global space is hierarchical and conditional:

- source selection and normalization;
- chunker family and parameters;
- representation kinds and generation prompts;
- embedding model, dimensionality, normalization, and task prefix;
- index backend and search parameters;
- query rewriting, routing, filters, retrieval depths, fusion, reranking;
- evidence admission and context layout;
- generation model, prompt, schema, citation protocol;
- tool descriptions, tool policy, loop limits, and finalization;
- evaluator and execution policy.

Attempting to mutate all of these at once yields a large, sparsely observed, non-stationary space with weak causal attribution. The framework should support the whole space, while studies normally use:

- one-target proof trials;
- block-coordinate optimization by subsystem;
- periodic interaction/factorial rounds;
- multi-fidelity screens before full validation;
- final joint Pareto selection.

6 5. Design principles for the foundation

6.1 5.1 One semantic source of truth

A study specification must be the authoritative declaration of:

- systems and component configurations;
- mutable targets and domains;
- datasets and split manifests;
- provider/model identities;
- budgets and fidelities;
- metrics and judge identities;
- objectives, constraints, and promotion policy.

The compiler emits canonical JSON IR, lockfiles, copied assets, and runtime configuration. Applications may expose command-line overrides only for values explicitly declared operational rather than semantic. Any override that changes results must create a new plan digest.

This rule eliminates the CoinVault runtime-contract/code-constant disagreement and the RAG-TTC generated-YAML/application-default split.

6.2 5.2 Separate semantics, execution, and evidence identities

Three fingerprints are needed:

1. **Semantic plan identity** - graph, component implementation/version, semantic configuration, prompts, schemas, models, and input artifact digests.

2. **Execution identity** - semantic plan plus workers, batching, retry policy, placement, runtime image, hardware, and provider endpoint class.
3. **Evidence identity** - study, datasets, evaluator/judge, schedule, candidate, plan, execution environment, and artifact manifests.

Cache keys should normally use semantic identity and exact inputs. Trial comparison and latency/cost claims use execution identity. Promotion decisions use evidence identity. Collapsing these identities either prevents safe cache reuse or produces false equivalence.

6.3 5.3 Compile before executing

No provider call or index build should occur until compilation has:

- resolved all component versions;
- validated schemas and typed ports;
- evaluated conditional activation;
- checked capability and effect requirements;
- verified assets and digests;
- constructed the build/query dependency DAG;
- derived cache keys and expected artifact roles;
- generated the persisted paired schedule;
- performed resource/cost preflight;
- frozen the evaluator and promotion policy.

The compiled plan should be inspectable, diffable, and replayable without the original authoring syntax.

6.4 5.4 Product semantics remain outside the generic kernel

The kernel should know that an output has schema `rag.evidence-set/v1`; it should not know what a CoinVault comparison intent means or which TTC plant attributes are required. Product packages own:

- source extraction and authorization boundaries;
- domain-specific tool implementations;
- product answer contracts and required facets;
- domain metrics and failure taxonomies;
- deployment activation and rollback.

They expose those semantics through versioned plugin descriptors, schemas, evaluators, and trace assertions. This is a ports-and-adapters boundary, not an attempt to erase domain meaning.

6.5 5.5 Optimize immutable plans, not mutable running systems

An optimizer proposes a patch. The compiler creates a new immutable plan. The executor evaluates it. Promotion produces a non-applying activation plan or repository change. The optimizer never edits production configuration in place.

6.6 5.6 Rich feedback and scalar decisions have different roles

Natural-language traces, contrasts, and diagnostics are useful for proposing changes. Deterministic metrics, blinded judgments, statistical comparisons, and hard gates authorize progression. A reflective model may suggest; it must not be the sole promotion authority.

6.7 5.7 Every mutation needs an activation contract

A candidate is invalid if the intended change was not present or was not capable of affecting the evaluated cases. Each mutable target should support:

- a compile-time application check;
- a runtime activation assertion;
- optional case-level exercise assertions;
- invariant assertions for protected components.

CoinVault has proved the value of this distinction. It should become generic framework vocabulary.

7 6. Target architecture

7.1 6.1 Layered view

```
+-------------------------------------------------------------------------+
| Promotion and governance                                                |
| constraints | non-inferiority | Pareto selection | audit | canary     |
+-------------------------------------------------------------------------+
| Search and learning                                                     |
| manual | Sobol | TPE/SMAC | Hyperband/BOHB | NSGA-II/MOBO | GEPA      |
+-------------------------------------------------------------------------+
| Evaluation and analysis                                                 |
| metrics | deterministic checks | judges | statistics | diagnostics     |
+-------------------------------------------------------------------------+
| Experiment control (ragopt)                                             |
| study | campaign | trial | cells | attempts | budgets | resume        |
+-------------------------------------------------------------------------+
| Compiler and plan kernel                                                |
| DSL -> typed IR | registry | effects | identity | dependency planner |
+-------------------------------------------------------------------------+
| RAG mechanisms (ragkit + product plugins)                               |
| ingest | chunk | represent | embed | index | retrieve | generate      |
+-------------------------------------------------------------------------+
| External infrastructure                                                 |
| CAS/S3 | SQLite/Parquet | queues/Batch/River | providers | telemetry |
+-------------------------------------------------------------------------+
```

The layer boundaries are deliberate:

- `ragkit` remains usable without `ragopt`.
- `ragopt` executes any compiled system, not only RAG.
- search algorithms consume a generic study/result view.
- distributed scheduling is an adapter behind `ExecutorBackend`; it is not embedded into the semantic kernel.
- product applications register components and studies, but do not fork experiment mechanics.

7.2 6.2 Core object model

```
StudySpec
  |-- base SystemSpec(s)
  |-- SearchSpace
  |-- DatasetSet
  |-- MetricSet
  |-- Objective/Constraint Policy
  |-- Fidelity Ladder
  |-- Search Strategy
```

```
  `-- Promotion Policy

Compiler
  `-- Plan + Lockfile + MutationCatalog + CostEstimate

Campaign
  |-- Trial 1 -> Plan/Patch -> Cells -> Attempts -> Observations
  |-- Trial 2 -> Plan/Patch -> Cells -> Attempts -> Observations
  `-- Archive -> feasible/Pareto/failed/rejected

Decision
  `-- reject | continue | promote-to-next-fidelity | review | activate-plan
```

The current `ragopt` candidate bundle maps naturally to a strict subset:

```
parent Snapshot      -> parent Plan
child Snapshot       -> compiled Plan(parent + Patch)
exactly one asset    -> Patch with max_changed_targets: 1
suite                -> Dataset split
paired eval cells    -> Trial cells
comparison + gate    -> Decision policy
```

This provides a compatibility path rather than a flag day.

7.3 6.3 Recommended repository ownership

A concrete package split could be:

```
ragkit/
  rag/...                existing domain types and implementations
  component/             descriptor adapters for ragkit components
  schema/                standard RAG port schemas
  trace/                 standard RAG domain payloads

ragopt/
  pkg/spec/              authoring models: StudySpec, SystemSpec, SearchSpace
  pkg/ir/                canonical typed Plan and Patch IR
  pkg/compiler/          resolution, validation, lowering, lockfiles
  pkg/registry/          component/evaluator/optimizer/policy registries
  pkg/artifact/          content-addressed refs and store interfaces
  pkg/runtime/           plan executor and node lifecycle
  pkg/study/             campaign/trial/cell/attempt state model
  pkg/trace/             generic span/event envelope
  pkg/eval/              scheduling, metric execution, paired evaluation
  pkg/stats/             bootstrap, intervals, sequential decisions
  pkg/search/             ask/tell interface and built-in strategies
  pkg/gate/               constraints and promotion policies
  pkg/review/             blinded review and aggregation
  pkg/runstore/           append-only local run implementation
  pkg/executor/           local plus external job backend interfaces
  pkg/plugin/             subprocess/WASI protocol clients
  schemas/                versioned JSON Schema/CUE definitions
  protocol/               RPC and envelope definitions
  conformance/            reusable plugin and store tests

coinvault/
  internal/ragplugins/...  product sources, tools, contracts, metrics
  studies/...              CoinVault StudySpec files
```

```
rag-ttc/
  pkg/ragplugins/...       TTC sources, tools, contracts, metrics
  studies/...              TTC StudySpec files
```

The `ragopt` production-refresh design correctly states that AWS Batch, River, or another job system should own durable worker execution. The proposed `pkg/executor` is therefore a port and state adapter, not a new queue product.

8 7. The compositional kernel and canonical IR

8.1 7.1 Why an IR is the missing center

The current code has two composition mechanisms:

- typed Go interfaces and `flow.Step` values in `ragkit`;
- arbitrary asset snapshots plus product-owned `Arm` implementations in `ragopt`.

Neither is a complete optimizer-facing program representation. A canonical IR provides:

- one stable object to hash, diff, patch, validate, execute, and archive;
- typed node and port compatibility;
- explicit build/query stage boundaries;
- legal mutation locations and domains;
- effect and capability declarations;
- deterministic dependency closure and reuse;
- static treatment checks;
- a foundation for cost modeling and plan exploration.

This is consistent with the recent RAG-Stack direction: an intermediate representation, cost model, and plan explorer separate the RAG design space from runtime deployment details. The proposed design is broader because it includes prompts, tools, evaluators, experiment custody, and promotion rather than only serving quality/performance.

8.2 7.2 Node model

A node should contain data, not executable function pointers:

```go
type Node struct {
    ID          NodeID
    Component   ComponentRef
    Config      json.RawMessage
    Inputs      map[string]PortBinding
    Outputs     map[string]SchemaRef
    Policy      ExecutionPolicyRef
    Annotations map[string]string
}

type ComponentRef struct {
    ID          string // e.g. ragkit.chunk.heading-aware
    Version     string // semantic component version
    Plugin      ArtifactRef
    Descriptor  string // descriptor digest
}
```

A descriptor supplies behavior metadata:

```go
type Descriptor struct {
    ID            string
    Version       string
```

```
    Inputs        []PortDescriptor
    Outputs       []PortDescriptor
    ConfigSchema  SchemaRef
    Determinism   DeterminismClass
    Effects       []Effect
    Capabilities  []string
    IdentityRules []IdentityRule
    TraceSchemas  []SchemaRef
}
```

Standard node families should include:

- source snapshot and extraction;
- parse/normalize;
- chunk;
- representation generation;
- embedding;
- lexical/vector/content index build and publication;
- query transformation and routing;
- retrieve/filter/fuse/collapse/hydrate;
- rerank and context admission;
- generation and structured output parsing;
- tools and bounded agent loops;
- answer contract validation;
- metric/judge nodes only in the evaluation graph.

The IR should not require that all implementations are RAG-specific. It should require schema-compatible ports.

8.3 7.3 Semantic port types

Raw Go types do not cross process/plugin boundaries. Ports use schema-qualified envelopes such as:

```
rag.document-set/v1
rag.chunk-set/v1
rag.representation-set/v1
rag.vector-set/v1
rag.index-bundle/v2
rag.query/v1
rag.hit-set/v1
rag.evidence-set/v1
rag.context/v1
rag.generation-request/v1
rag.answer/v1
rag.answer-contract-result/v1
rag.metric-set/v1
```

The envelope points to an artifact rather than embedding arbitrary large values:

```
type Envelope struct {
    Schema   SchemaRef
    Artifact artifact.Ref
    Metadata map[string]string
}
```

In-process adapters may decode envelopes into the existing `ragkit` types. External plugins receive the same logical protocol over RPC or standard I/O.

8.4 7.4 Pure nodes, effectful nodes, and explicit loops

The compiler should classify effects:

```
pure/local          parse, deterministic chunk, fusion, validation
content-cacheable   embedding, generation, reranking with exact identity
stateful-read       index query, content store read
stateful-write      index build, artifact publication
network             provider/tool calls
secret-bearing      authorized SQL/tool execution
non-deterministic   unseeded provider generation, live data reads
```

These declarations enable security policy, cache rules, scheduling, and reproducibility labels.

Ordinary graph cycles should be rejected. Agent/tool iteration should use an explicit bounded `Loop` node with:

- state schema;
- maximum iterations/provider calls;
- allowed tools;
- termination predicate;
- finalization reserve policy;
- per-iteration trace schema;
- resource ceiling.

This keeps the graph analyzable without pretending an agent loop is an acyclic function.

8.5 7.5 Compiler passes

A practical compiler pipeline is:

1. Parse strict Study/System specifications with known fields only.
2. Resolve includes, assets, component aliases, and registry versions.
3. Validate component configuration schemas.
4. Type-check graph ports and required/optional inputs.
5. Validate effects, secrets, network, and authorization policy.
6. Expand approved macros into ordinary nodes.
7. Evaluate conditional activation and construct the concrete graph.
8. Separate build, serving, and evaluation subgraphs.
9. Compute semantic identities bottom-up from component, config, and input identities.
10. Discover mutable targets and validate search-domain compatibility.
11. Compute dependency closures and candidate artifact reuse groups.
12. Construct cost/resource preflight from component estimators.
13. Generate a persisted schedule and randomization seed.
14. Emit canonical Plan JSON, Lockfile, MutationCatalog, and diagnostics.

Canonicalization must specify map ordering, numeric normalization, path normalization, omitted/default field rules, and schema versions. Semantic and byte digests should both be retained, as newer `ragopt` already does for several artifacts.

8.6 7.6 Identity and cache rules

For node v:

$$ID_v = H(component_v, version_v, semanticConfig_v, inputArtifactDigests_v, relevantEnvironment_v)$$

Execution policy such as workers and retries must not enter ID_v unless it changes semantic output. This preserves the `flow.Step` principle. However, provider parameters that affect output - temper-

ature, reasoning mode, tool policy, task prefix, schema, prompt, endpoint semantics - must enter the identity.

The compiler should generate identity rules from descriptors rather than relying on each application to remember relevant fields. It should fail closed when an effectful component cannot provide a complete identity contract.

Safe cross-candidate reuse then becomes automatic:

```
Candidate A: same chunk/embed/index, changed answer prompt
Candidate B: same chunk/embed/index, changed retrieval k
Candidate C: changed chunk size

A and B reuse the index artifact.
C rebuilds chunk descendants but may reuse source normalization.
```

Cache namespaces must include semantic node identity. Shared caches without complete candidate identity can mask treatments; fully isolated caches waste build and provider work. Content-addressed subgraph reuse is the correct middle path.

8.7 7.7 Patches and mutation targets

A candidate should be a typed patch against a parent plan:

```
type Patch struct {
    APIVersion string
    ID         string
    ParentPlan Digest
    Operations []Operation
    Proposer   Proposer
    Hypothesis Hypothesis
    Evidence   []EvidenceRef
}

type Operation struct {
    Target MutationTargetRef
    Kind   OperationKind // replace, set, insert, remove, select
    Value  json.RawMessage
}
```

A mutation target declares its legal domain and proof obligations:

```
type MutationTarget struct {
    ID              string
    Path            string
    Domain          Domain
    ActiveWhen      Expr
    Invariants      []Expr
    Activation      []TraceAssertion
    Exercise        []TraceAssertion
    SecurityClass   string
    RebuildBoundary NodeID
}
```

Protocol modes can constrain patches:

```
proof:      exactly one changed target; strict invariants
screen:     bounded target block; cheap fidelity
joint:      multiple targets allowed; interaction-aware analysis
structural: graph operations allowed; strongest validation
```

This preserves the scientific discipline of current `ragopt` while permitting controlled expansion.

9 8. DSL design

9.1 8.1 Authoring format

Use a declarative format with a strict schema. YAML is accessible for humans; JSON is canonical; CUE is attractive for validation and composition. The essential decision is not the surface syntax. It is that all surfaces compile to the same IR.

Recommended tools by role:

- YAML/JSON/CUE for StudySpec and SystemSpec.
- CEL-like expressions for predicates, constraints, activation, and gates.
- SQL for read-only analysis views over the derived warehouse.
- Starlark, Python, or shell only as explicit `script` components with digest-pinned code, declared inputs/outputs/effects, sandboxing, and no hidden mutation of framework state.

Do not begin with a Turing-complete workflow language. It would reproduce application code in a less testable form and make identity analysis undecidable in practice.

9.2 8.2 Illustrative StudySpec

```yaml
api_version: ragopt.dev/v1alpha1
kind: Study
metadata:
  name: ttc-joint-rag-screen

system:
  base: systems/ttc-production.yaml

mutable:
  mode: block
  max_changed_targets: 3
  targets:
    - id: chunk-size
      path: nodes.chunk.config.maximum_runes
      domain: {type: int, min: 600, max: 2200, step: 200}

    - id: representation-kinds
      path: nodes.represent.config.kinds
      domain:
        type: choice
        values:
          - [raw]
          - [raw, breadcrumbs]
          - [raw, breadcrumbs, generated_questions]

    - id: retrieve-k
      path: nodes.retrieve.config.retrieve_k
      domain: {type: int, values: [10, 20, 40, 80]}

    - id: reranker
      path: nodes.rerank.component
      domain:
        type: choice
        values:
          - ragopt.builtin/disabled@v1
          - ragkit.rerank/term-overlap@v1
          - provider.rerank/nomic@v2
```

```yaml
      active_when: "nodes.retrieve.config.retrieve_k >= 20"

    - id: answer-prompt
      path: nodes.generate.config.prompt_asset
      domain:
        type: text
        proposer: gepa
        max_bytes: 12000
        forbidden_classes: [credentials, audit_answers, judge_instructions]

fidelities:
  - id: smoke
    cases: datasets/diagnostic-smoke.json
    repeats: 1
    judges: [deterministic]
  - id: feedback
    cases: datasets/feedback-v3.json
    repeats: 1
    judges: [deterministic, luna]
  - id: validation
    cases: datasets/validation-v3.json
    repeats: 2
    judges: [deterministic, luna, blinded_pairwise]
  - id: audit
    cases: datasets/audit-v2.json
    repeats: 2
    judges: [deterministic, luna, human_sample]

metrics:
  - rag.retrieval/required_chunk_recall@v1
  - rag.answer/faithfulness@v3
  - rag.answer/facet_coverage@v2
  - rag.answer/abstention_f1@v1
  - rag.runtime/provider_tokens@v1
  - rag.runtime/p95_latency_ms@v1
  - rag.index/build_seconds@v1
  - rag.index/bytes@v1

objectives:
  pareto:
    maximize: [faithfulness, facet_coverage, abstention_f1]
    minimize: [provider_tokens, p95_latency_ms, build_seconds, index_bytes]

constraints:
  - "critical_safety_failures == 0"
  - "contract_valid_rate >= 0.995"
  - "provider_failure_rate <= 0.01"
  - "required_chunk_recall >= 0.95"

search:
  strategy: portfolio
  phases:
    - {optimizer: sobol, fidelity: smoke, trials: 48}
    - {optimizer: bohb, fidelity: feedback, promoted: 12}
    - {optimizer: nsga2, fidelity: validation, generations: 4}
    - {optimizer: gepa, fidelity: feedback, target: answer-prompt, trials: 8}

promotion:
```

```yaml
  require_fidelity: audit
  primary:
    metric: facet_coverage
    minimum_paired_delta: 0.02
    confidence: 0.95
  non_inferiority:
    faithfulness: -0.005
    abstention_f1: -0.01
  require_human_review: true
  emit_activation_plan: true
```

This example is intentionally broad. A normal proof study can set `mode: proof` and one target.

9.3 8.3 SystemSpec

A system file describes graph topology and components, not evaluation policy:

```yaml
api_version: ragopt.dev/v1alpha1
kind: System
metadata: {name: ttc-production}

nodes:
  source:
    component: ttc.source/catalog-snapshot@v2
    config: {manifest: assets/catalog-snapshot.json}

  chunk:
    component: ragkit.chunk/heading-aware@v2
    inputs: {documents: source.documents}
    config:
      maximum_runes: 1400
      overlap_runes: 120
      minimum_section_runes: 180

  represent:
    component: ragkit.represent/multi@v2
    inputs: {chunks: chunk.chunks}
    config:
      kinds: [raw, breadcrumbs]
      prompt_set: assets/representation-prompts.yaml

  embed:
    component: provider.embed/nomic@v1
    inputs: {representations: represent.representations}
    config:
      model: nomic-embed-text-v1.5
      dimensions: 768
      document_prefix: "search_document:"
      query_prefix: "search_query:"

  index:
    component: ragkit.index/hybrid-bundle@v2
    inputs:
      chunks: chunk.chunks
      representations: represent.representations
      vectors: embed.vectors
    config: {lexical: bleve-v2, vector: sqlite-exact-v1}
```

```
  retrieve:
    component: ragkit.retrieve/weighted-rrf@v2
    inputs: {index: index.bundle, query: $request.query}
    config: {retrieve_k: 40, rrf_constant: 60}

  generate:
    component: ttc.agent/tool-loop@v3
    inputs: {query: $request.query, retrieval: retrieve.hits}
    config:
      prompt_asset: assets/orchestration.txt
      answer_schema: assets/answer-schema.json
      maximum_provider_calls: 4
      tools: [search]

outputs:
  answer: generate.answer
  trace: generate.trace
```

The compiler resolves every component and asset into exact digests and emits the runtime configuration. RAG-TTC no longer constructs `tool-qa.yaml` through a string literal; CoinVault no longer branches on treatment type to assemble its runner.

9.4 8.4 Patch example

A GEPA-style proposer should emit structured data, not an executable diff:

```
api_version: ragopt.dev/v1alpha1
kind: Patch
metadata:
  id: gepa-answer-prompt-017
parent_plan: sha256:...
proposer:
  kind: gepa
  identity: reflector/luna@prompt-sha256:...
hypothesis:
  statement: >-
    Comparison failures occur because the agent answers after one broad search.
    Requiring separate subject searches should improve facet coverage.
  expected_metrics: [facet_coverage]
  regression_risks: [provider_calls, latency, over-search]
evidence:
  cases: [compare-014, compare-031, compare-044]
operations:
  - target: answer-prompt
    kind: replace_asset
    value:
      media_type: text/plain
      path: proposal/orchestration.txt
      sha256: sha256:...
```

The compiler computes the actual changed targets independently, just as current `ragopt` independently verifies changed asset bytes.

9.5 8.5 Lockfile

The lockfile should include:

- resolved component versions and plugin digests;
- canonical config digests;

- input asset byte and semantic digests;
- provider/model protocol identities;
- schemas and metric/judge versions;
- compiler/runtime versions;
- environment assumptions;
- generated schedule digest;
- all derived budget ceilings;
- expected output roles and cache identities.

It should be generated, never hand-maintained.

10 9. Extension and plugin model

10.1 9.1 Three extension tiers

Use different mechanisms for different trust and deployment requirements.

10.1.1 Tier 1: statically linked Go registrations

Best for trusted `ragkit` and product components. It provides type safety and low overhead. Applications import packages and call `Register` during startup.

10.1.2 Tier 2: subprocess RPC plugins

Best for independently versioned Python/Go/Rust components and provider adapters. Use gRPC/Connect or a framed standard-I/O protocol with schema-qualified envelopes. The plugin process exposes:

```
Describe
ValidateConfig
Estimate
Execute
Health
Shutdown
```

The host controls environment, working directory, network, secrets, timeouts, output size, and artifact paths.

10.1.3 Tier 3: WASI/sandboxed plugins

Best for untrusted or third-party deterministic transforms and metrics. Capabilities are explicit; network and filesystem access are denied by default.

Do not use Go's native `plugin` package as the public extension boundary. Its toolchain and dependency ABI coupling works against the exact versioning and reproducibility goals of this framework.

10.2 9.2 Registry interfaces

At minimum, maintain registries for:

- components;
- schemas/codecs;
- metrics and evaluators;
- judges;
- optimizers/proposers;
- gate/promotion policies;
- artifact stores;

- executor backends;
- report/export formats.

Registration must reject duplicate `(ID, version)` pairs and require immutable descriptors.

10.3 9.3 Component execution contract

A generic component factory can look like:

```
type ComponentFactory interface {
    Descriptor() Descriptor
    ValidateConfig(ctx context.Context, raw json.RawMessage) error
    Estimate(ctx context.Context, req EstimateRequest) (Estimate, error)
    New(ctx context.Context, raw json.RawMessage,
        deps RuntimeDependencies) (Component, error)
}

type Component interface {
    Execute(ctx context.Context, req ExecuteRequest,
        emit TraceEmitter) (ExecuteResult, error)
    Close(ctx context.Context) error
}
```

`ExecuteRequest` supplies only assigned artifact paths, input envelopes, cell identity, budget handles, and scoped secrets. Components never receive unrestricted run-root access. This generalizes the newer `ragopt` evidence guard.

10.4 9.4 Evaluator contract

Metrics should be independent plugins with explicit input levels:

```
type MetricDescriptor struct {
    ID          string
    Version     string
    Direction   Direction
    Unit        string
    Level       Level // node, retrieval, answer, cell, trial, campaign
    Inputs      []SchemaRef
    Missingness MissingnessPolicy
    Aggregation AggregationSpec
}

type Evaluator interface {
    Descriptor() MetricDescriptor
    Evaluate(context.Context, EvaluationInput,
        TraceEmitter) (MetricObservation, error)
}
```

A judge is an evaluator with provider effects, a prompt/schema identity, and a separate evaluation budget. A product-specific deterministic contract validator uses the same interface without provider effects.

10.5 9.5 Optimizer ask/tell contract

Search engines should not depend on product code:

```
type Optimizer interface {
    Descriptor() OptimizerDescriptor
    Initialize(context.Context, StudyView) error
```

```
    Ask(context.Context, AskRequest) ([]PatchProposal, error)
    Tell(context.Context, []TrialObservation) error
    Snapshot(context.Context) (artifact.Ref, error)
    Restore(context.Context, artifact.Ref) error
}
```

`StudyView` exposes the search space, immutable objectives/constraints, available fidelities, and summarized archive. Rich textual proposers may request bounded diagnostic packets through a separate `FeedbackProvider` rather than direct access to audit artifacts.

10.6 9.6 Conformance suites

Every extension must pass reusable tests for:

- descriptor stability and schema validity;
- canonical identity completeness;
- deterministic replay claims;
- artifact path confinement;
- cancellation and timeout behavior;
- usage preservation on errors;
- maximum-output enforcement;
- trace schema validity and sequence ordering;
- no undeclared network/secret access;
- cache hit/miss equivalence;
- crash and retry behavior.

A plugin should not be admitted into optimization campaigns merely because it implements an interface.

11 10. Experiment, artifact, and state model

11.1 10.1 Immutable hierarchy

Use a stable hierarchy:

```
Study
  `-- Campaign
        `-- Trial
              `-- Cell
                    `-- Attempt
                          `-- NodeExecution / Span
```

- **Study**: immutable evaluation and search contract.
- **Campaign**: adaptive search history under that study.
- **Trial**: one compiled candidate plan at one fidelity.
- **Cell**: one case/repeat/arm/environment block.
- **Attempt**: one retryable execution with explicit predecessor.
- **NodeExecution**: one component invocation or cache result.

Current `ragopt` runs correspond roughly to one two-arm trial. Generalization should retain the current cell format as a compatibility projection.

11.2 10.2 Event sourcing and projections

The authoritative state should be append-only events plus immutable artifacts. Typical events are:

```
study.compiled
campaign.created
```

```
patch.proposed
trial.admitted
trial.rejected_preflight
cell.scheduled
attempt.started
node.started
node.cache_hit
node.completed
metric.recorded
attempt.failed
attempt.completed
cell.committed
trial.aggregated
gate.evaluated
candidate.promoted_fidelity
review.requested
campaign.completed
activation.plan_emitted
```

Each event includes schema version, sequence, timestamp, actor, study/campaign/trial/cell/attempt IDs, previous digest, event digest, and payload artifact reference. The v0.0.1 cell chain is a useful starting point; extend the tamper-evident pattern to campaign events without forcing all domain data into the common record.

A local JSONL journal can remain the first implementation. SQLite or Parquet is a **projection** for queries and dashboards, rebuildable from events and artifacts. This aligns with the RAG-TTC design and avoids making ad-hoc analyst state authoritative.

11.3 10.3 Idempotency and resume

Every schedulable unit has an idempotency key derived from:

```
study digest
trial/plan digest
case digest
repeat and block
fidelity
execution identity
node semantic identity and input digests
```

Resume rules must distinguish:

- committed result: verify and reuse;
- complete artifact but uncommitted journal: reconcile and commit if valid;
- active lease with heartbeat: do not duplicate;
- expired attempt: create a new attempt linked to the previous one;
- uncertain provider usage: preserve billed usage and conservatively charge budget;
- corrupt or mismatched artifact: quarantine and rerun only under explicit policy.

The application should never reconstruct budget state from informal logs. Budget usage is a first-class event and node observation.

11.4 10.4 Content-addressed artifact store

Generalize `runstore` paths into an `artifact.Store` interface, with local filesystem first and S3-compatible storage later:

```
type Ref struct {
    Digest    string
    SizeBytes int64
```

```
    MediaType string
    Schema    string
    Role      string
}

type Store interface {
    Put(context.Context, io.Reader, PutOptions) (Ref, error)
    Open(context.Context, Ref) (io.ReadCloser, error)
    Verify(context.Context, Ref) error
    Link(context.Context, Ref, LogicalPath) error
}
```

Artifacts include source snapshots, compiled plans, lockfiles, patches, index bundles, traces, answers, judge records, metric tables, reports, optimizer state, and activation plans. Logical run directories can be manifests of references, preserving today's inspectability while avoiding duplicated bytes.

11.5 10.5 Fingerprint matrix

Record separate fingerprints rather than one overloaded digest:

Fingerprint	Includes	Typical use
Source	corpus snapshot, connector query/version, normalization	data lineage and refresh decisions
Build plan	offline graph and semantic configs	index reuse and invalidation
Index artifact	bundle manifest and payloads	serving and retrieval identity
Query plan	online graph, prompts, tools, model protocol	answer cache and comparison
Evaluator	metric/judge code, prompts, schemas, calibration data	result comparability
Study	datasets, objectives, constraints, fidelities, schedule policy	campaign identity
Execution	runtime image, hardware, workers, retries, endpoint class	latency/cost reproducibility
Evidence	all of the above plus schedule and artifacts	audit and promotion

11.6 10.6 Reproducibility tiers

Not all provider-backed RAG is bit-reproducible. Declare tiers:

1. **Exact** - deterministic local code and verified cached outputs reproduce bytes.
2. **Replayable** - provider outputs are immutable artifacts; the run can be reanalyzed exactly but not necessarily regenerated.
3. **Statistically reproducible** - equivalent identity and repeated protocol produce comparable distributions.
4. **Operationally comparable** - live data or provider drift is present; the environment/time window is part of evidence.

Promotion policies can require a minimum tier per metric. Safety and deterministic contract checks should prefer exact/replayable evidence.

12 11. Unified trace and treatment model

12.1 11.1 Generic envelope

CoinVault's trace is rich because it observes provider starts/finishes, tool calls/results, evidence, semantic IDs, limits, reranking, prompt digests, intermediate errors, terminal states, and projection errors. RAG-TTC retains session/agent traces. The foundation should not discard this richness, but it should normalize the outer envelope:

```
type Event struct {
    APIVersion string
    Sequence   uint64
    At         time.Time
    StudyID    string
    CampaignID string
    TrialID    string
    CellID     string
    AttemptID  string
    NodeID     string
    Kind       string
    Status     string
    Duration   time.Duration
    Usage      Usage
    Error      *Failure
    Payload    SchemaPayload
}

type SchemaPayload struct {
    Schema   SchemaRef
    Artifact artifact.Ref
}
```

Domain payloads remain versioned and product-owned. Example kinds:

```
component.started
component.completed
provider.call.started
provider.call.completed
tool.call.requested
tool.call.completed
retrieval.channel.ranked
retrieval.fused
context.admitted
generation.completed
contract.validated
metric.observed
```

This gives generic schedulers and analysts common fields without flattening domain semantics into strings.

12.2 11.2 Spans and causal links

Use an OpenTelemetry-inspired hierarchy, while retaining durable artifact references:

```
cell span
  |- query-transform span
  |- retrieval span
  |    |- lexical span
  |    |- vector span
  |    `- fusion span
```

```
  |- rerank span
  |- context span
  `- agent-loop span
       |- provider call 1
       |- tool call 1 -> retrieval child link
       `- provider call 2
```

Each output records its input artifact refs and parent/linked spans. This supports failure attribution and treatment exercise queries.

12.3 11.3 Generic treatment assertions

Move CoinVault's treatment checks into declarative assertions evaluated over plan and trace:

```yaml
activation:
  - expr: "plan.nodes.retrieve.config.retrieve_k == 40"
  - expr: "trace.exists(kind == 'retrieval.started' && node_id == 'retrieve')"

exercise:
  - expr: "trace.metric('retrieval.returned_items') > 20"
  - expr: "trace.attr('rerank.applied') == true"

invariants:
  - expr: "candidate.index.digest == incumbent.index.digest"
  - expr: "candidate.evaluator.digest == incumbent.evaluator.digest"
```

Assertions are schema-aware and type-checked at compile time. Product plugins can register functions such as `required_facets_covered` or `comparison_subject_count`, but the execution and reporting protocol remains generic.

Distinguish four statuses:

- **not applied** - patch did not reach the compiled plan;
- **not activated** - runtime did not load or expose it;
- **not applicable** - case could not exercise the treatment;
- **applicable but not exercised** - runtime path was available but behavior did not use it.

Metrics from invalid treatment cells should not be interpreted as evidence of no effect.

12.4 11.4 Trace privacy and size

Traces can contain prompts, private corpus text, tool results, SQL data, and reasoning. Descriptors must declare redaction and retention policy. Store large payloads as encrypted artifacts; keep summaries and hashes in events. Provide field-level classifications such as public, internal, customer data, secret, and evaluator-only. Optimizers receive least-privilege diagnostic views, not raw unrestricted traces.

13 12. Evaluation foundation

13.1 12.1 Metric taxonomy

The framework should support metrics at multiple levels.

13.1.1 Offline/index metrics

- source coverage and extraction errors;
- chunk validity, size distribution, overlap, fragmentation, and duplication;
- representation count, generation failure, and representation-to-chunk lineage;

- embedding calls, tokens, cache reuse, dimensionality, and drift checks;
- build duration, peak memory, index bytes, publication/verification time;
- incremental refresh reuse and changed-descendant count.

13.1.2 Retrieval metrics

- document/chunk/representation recall at k;
- MRR, nDCG, precision, and required-source recall;
- facet/evidence coverage;
- distractor displacement and duplicate concentration;
- channel contribution, fusion stability, and reranker lift;
- evidence admission recall and context truncation loss;
- query-transform validity and semantic preservation.

13.1.3 Answer metrics

- task correctness and required facet coverage;
- faithfulness/claim support;
- citation precision, recall, validity, and coverage;
- answer relevance and completeness;
- structured contract validity;
- appropriate abstention, clarification, and scope disclosure;
- robustness across paraphrase, ambiguity, and temporal cases.

13.1.4 Agent/tool metrics

- correct tool selection and route;
- argument/schema validity;
- authorization and scope compliance;
- tool-result use and evidence provenance;
- loop termination/finalization;
- provider/tool call count and avoidable calls;
- intermediate failure recovery;
- treatment activation and exercise.

13.1.5 System metrics

- end-to-end success/failure by attributable class;
- p50/p95/p99 latency and timeouts;
- input/output/reasoning/cached tokens;
- provider/tool/embedding/rerank calls;
- measured monetary cost;
- CPU, memory, storage, and network use;
- cache hit rate and warm/cold behavior.

13.2 12.2 Metric descriptors and missingness

Every metric needs:

```
ID and version
direction: maximize/minimize/target
unit and valid range
observation level
required input schemas
aggregation rule
case-group behavior
missingness policy
```

`failure interaction`
`confidence procedure`

For example, faithfulness may be undefined for a failed contract or valid abstention. That is different from faithfulness zero. A gate may require all answerable cases to have faithfulness and separately constrain failure and abstention rates.

Metric version changes invalidate direct comparisons unless an explicit migration/recomputation path exists. The warehouse should keep raw observations and derived aggregates.

13.3 12.3 Dataset layers

Use at least four logically separate datasets:

1. **Feedback/development** - diagnostic examples, trajectories, and judge text visible to proposers.
2. **Validation** - paired decision evidence visible only as bounded results after proposal.
3. **Audit/holdout** - protected final gate; not used to guide iterative search.
4. **Production probes** - sampled, privacy-reviewed operational cases used to detect drift and nominate future diagnostic cases.

RAG-TTC already articulates a compatible three-layer evaluation model. The addition of a separately named feedback set makes proposer access explicit.

Manifests should include stable case IDs, groups, input digests, labels/answerability, protected fields, and split assignment. Rotating or temporal audit sets reduce long-term benchmark overfitting. A campaign must never move cases between splits without a new study identity.

13.4 12.4 Deterministic checks before judges

Run cheap and exact checks first:

- schema/parse/contract validity;
- source/citation existence and lineage;
- treatment activation;
- required protected identities;
- forbidden tool/authorization events;
- retrieval labels and facet coverage where available;
- budget and resource ceilings;
- duplicate/corrupt artifacts;
- failure taxonomy.

Only judge eligible cells. This saves budget and prevents invalid answers from receiving misleading quality scores.

13.5 12.5 Judge governance

An LLM judge is a versioned effectful component. Freeze:

- model/provider protocol;
- prompt and output schema;
- statement extraction policy;
- evidence presented;
- temperature/reasoning settings;
- cache rules;
- missingness and retry policy.

Use blinded randomized pairwise judgments where absolute scores are unstable. Keep variant identity outside the reviewer/judge payload when possible, following `ragopt/review`. Calibrate judge

behavior against human-labeled samples and report disagreement. A judge supplies evidence; promotion policy owns the decision.

The proposer and evaluator are separate roles even when backed by the same model family. The proposer must not receive validation/audit answers, judge instructions, or hidden labels.

13.6 12.6 Statistical comparison

A minimum statistical layer should provide:

- paired means/medians and per-case deltas;
- hierarchical bootstrap over cases and repeats;
- confidence intervals for rates and continuous metrics;
- group/worst-group summaries;
- non-inferiority tests for protected metrics;
- effect sizes, not only p-values;
- failure and missingness counts;
- arm-order and environment-block diagnostics.

For adaptive campaigns, add one of:

- a protected final audit set used once per promotion candidate;
- alpha-spending or always-valid confidence sequences;
- nested validation where the search archive is selected on development and confirmed on audit.

Avoid ordinary uncorrected tests after selecting the best of dozens of candidates. The selection process changes the error rate.

13.7 12.7 Pareto and policy views

Maintain two complementary views:

1. **Feasible Pareto archive** - all non-dominated candidates under declared objectives and constraints.
2. **Product operating policy** - selects a preferred point or region, for example minimum cost subject to quality floors, or maximum quality under p95 latency and budget ceilings.

A single weighted sum hides useful trade-offs and is unstable when units change. The current hard-gate plus target/tie-break structure is a good promotion policy for focused proof experiments; it should not be the only campaign representation.

14 13. Search and learning plane

14.1 13.1 Search space classes

The framework must distinguish domains because different algorithms apply:

- continuous and integer parameters;
- categorical choices;
- conditional/hierarchical parameters;
- sets and ordered lists;
- text assets;
- structured JSON/YAML assets;
- graph component choices;
- structural insert/remove/rewire operations;
- execution policies and deployment choices.

A domain validates values, provides canonical encoding, declares neighborhood/mutation operators, and can estimate whether a change crosses a rebuild boundary.

14.2 13.2 Optimizer portfolio

No single optimizer is appropriate across all dimensions. Provide a portfolio:

Situation	Recommended method
Very small, hypothesis-driven change	Manual proposal plus proof protocol
Initial mixed-space screening	Random, Latin/Sobol, or fractional-factorial designs
Conditional numeric/categorical space	TPE or SMAC-style model-based search
Strong fidelity ladder and many cheap trials	Successive halving, Hyperband, or BOHB
Mixed-variable multi-objective population	NSGA-II or related evolutionary search
Expensive low/moderate-dimensional noisy objectives	Constrained MOBO such as qNEHVI-style acquisition
Text prompts/descriptions/policies	GEPA-style reflective proposal with structured patches
Graph structure	Constrained grammar/evolution after component-level search is stable

The first built-ins should be simple and testable: manual, grid/random/Sobol, successive halving, and a Pareto archive. TPE/BOHB and reflective text proposal can follow. Advanced MOBO is useful only after metric and campaign data are reliable.

14.3 13.3 Multi-fidelity evaluation

Fidelity is a vector, not just case count:

```
case subset and group coverage
number of repeats
judge depth: deterministic -> one judge -> pairwise/human
provider/model tier
index/corpus sample size
retrieval depth
trace detail
warm versus cold execution
```

A fidelity ladder must state which metrics are trustworthy at each level. Critical safety/authorization checks are never approximated away. Early stopping should rely on conservative bounds, not simply current mean score.

Index candidates need special scheduling:

1. compile candidate build subgraphs;
2. group identical build identities;
3. build each unique artifact once;
4. evaluate many query plans against each artifact;
5. promote only the promising index/query combinations to full corpus and audit.

This can reduce an apparent $N_{index} \times N_{query}$ build matrix to N_{index} builds plus query evaluations.

14.4 13.4 Block-coordinate campaign

A pragmatic all-dimensions workflow is:

```
Stage A: validate evaluator, traces, and baseline
Stage B: screen offline/index dimensions on retrieval diagnostics
Stage C: screen query/retrieval/context dimensions on fixed promising indexes
Stage D: optimize prompts and tool descriptions with rich trajectories
Stage E: tune execution policy and serving placement without changing semantics
Stage F: run a small interaction design across top blocks
Stage G: full paired validation and audit of the feasible Pareto set
```

Periodically re-open earlier blocks because interactions matter. For example, a better reranker may change the best retrieve depth; a new chunker may change the best prompt. Record interaction experiments explicitly rather than allowing uncontrolled simultaneous changes.

14.5 13.5 GEPA-style textual optimization

GEPA's important contribution for this design is not a brand-specific algorithm call. It is the loop:

```
rich trajectories -> targeted reflection -> complete textual replacement
-> evaluation -> retain complementary successful lessons
```

A framework-native reflection packet should contain:

- target component and current text;
- immutable component schema and size/security constraints;
- selected failure and successful-contrast cases from feedback only;
- retrieval, tool, contract, and judge traces in bounded form;
- per-case metric deltas and failure ownership;
- previous proposals and why they failed;
- declared objective and protected regressions.

The reflector returns a `PatchProposal` with one complete replacement, hypothesis, expected metrics, risks, and evidence case IDs. It does not edit files directly. Static validation and secret/injection scanning run before any evaluation.

Maintain ancestry and a textual lesson archive. GEPA's Pareto-oriented retention is useful because one prompt may improve comparison coverage while another improves abstention. A later synthesis proposal can combine lessons, but only under a multi-target protocol with fresh validation.

14.6 13.6 Cost model and plan exploration

A lightweight cost estimator is valuable even before sophisticated search:

- exact counts from graph/cardinality for deterministic steps;
- provider-call and token upper bounds from component descriptors;
- empirical latency/resource models by execution identity;
- index size/build estimates from sampled corpus statistics;
- cache/reuse estimates from content digests.

The estimator is advisory for pruning, batching, and admission. Measured artifacts remain authoritative. Track prediction error so the model itself can improve without changing quality evaluation.

15 14. Engineering workflow

15.1 14.1 Study-as-code workflow

A normal optimization change should proceed through repository artifacts:

1. **Study proposal** - add or modify a versioned StudySpec, datasets, metric declarations, budgets, and promotion policy.
2. **Compile in CI** - resolve the base SystemSpec and produce a canonical plan/lockfile diff.

3. **Baseline certification** - execute or verify the incumbent under the exact study and record baseline evidence.
4. **Campaign launch** - create an immutable campaign manifest and admitted budget.
5. **Candidate proposal** - manual or automated proposer emits typed patches and hypotheses.
6. **Static validation** - schema, security, identity, rebuild scope, and treatment assertions.
7. **Screening fidelity** - cheap deterministic and feedback evaluation.
8. **Adaptive promotion** - optimizer selects candidates for higher fidelity.
9. **Validation and audit** - paired, repeated, blinded, statistically analyzed.
10. **Promotion review** - generated report, plan diff, Pareto context, risks, human annotations.
11. **Activation plan** - non-applying deployment artifact or repository pull request.
12. **Canary and rollback** - deployment owner applies, monitors, and can atomically restore the incumbent.

The framework should make each step resumable and independently inspectable. Candidate generation is optional; a human-authored patch follows the same path.

15.2 14.2 Branch and review discipline

Separate four kinds of changes:

- framework/component code;
- evaluator or dataset changes;
- system configuration candidates;
- production activation.

Do not change the evaluator and candidate in the same evidence campaign. When evaluator improvements are necessary, establish a new study, recompute the incumbent, and start a new campaign identity.

Machine-generated candidates should live in campaign artifacts or a dedicated generated branch. Only promoted plans enter ordinary product configuration. The promotion pull request should contain:

- compiled semantic diff;
- source patch and proposer hypothesis;
- study, dataset, evaluator, and environment digests;
- per-case paired report and uncertainty;
- constraint/gate results;
- Pareto comparison;
- treatment activation/exercise report;
- human review summary;
- activation and rollback instructions.

15.3 14.3 CI tiers

15.3.1 Fast pre-commit/PR checks

- schema and compiler tests;
- canonicalization/golden plan tests;
- graph type/effect validation;
- plugin conformance tests with fixtures;
- deterministic metric and gate tests;
- no-network synthetic end-to-end study;
- static secret and unsafe-mutation scan.

15.3.2 Offline integration checks

- build/query DAG reuse;

- local content-addressed store;
- crash/resume and journal-chain verification;
- cache hit/miss equivalence;
- event projection rebuild;
- paired scheduling/randomization;
- report generation.

15.3.3 Provider-backed scheduled checks

- small smoke suite under explicit budget;
- judge calibration sample;
- provider protocol drift detection;
- selected high-fidelity candidate evaluation.

15.3.4 Manual/audit checks

- protected audit suite;
- human review;
- safety/authorization scenarios;
- production-like cold/warm latency and resource tests.

15.4 14.4 Failure handling workflow

Use a closed outer failure taxonomy with extensible product subtypes:

```
compile/configuration
asset/integrity
admission/budget
cache/artifact
provider
embedding/reranking
tool/authorization
timeout/cancellation
retrieval
contract/parse
judge/evaluator
trace/projection
infrastructure
unknown
```

A failure record includes owner, retryability, billed usage certainty, affected metric eligibility, and native artifact. Failed cells remain data. They are not dropped from denominators unless the metric explicitly defines otherwise.

15.5 14.5 Production feedback loop

Production telemetry should not directly train or mutate the system. Use a reviewed loop:

```
production probes and user feedback
        -> privacy/safety review
        -> deduplicated diagnostic candidates
        -> labeled feedback or future audit version
        -> new Study version
        -> ordinary campaign and promotion protocol
```

Keep production probes separate from the active audit set to prevent inadvertent leakage and shifting labels.

16 15. Security and governance

16.1 15.1 Threat model

A self-optimizing RAG system processes untrusted corpus text, model-generated patches, scripts/plugins, provider outputs, and potentially sensitive tool data. Threats include:

- prompt injection entering reflector packets;
- candidate prompts exfiltrating secrets or audit answers;
- malicious or accidental tool expansion;
- unsafe SQL or filesystem operations;
- plugin supply-chain changes;
- hidden network calls;
- artifact path traversal or symlink attacks;
- cache poisoning or cross-candidate contamination;
- evaluator tampering;
- budget exhaustion;
- production activation without review.

16.2 15.2 Controls

Required controls include:

- deny-by-default mutation policy; judge, audit, authorization, and secret paths are immutable;
- separate credentials and network allowlists by component effect;
- sandboxed external plugins and scripts;
- digest-pinned plugin binaries/images and signed provenance where available;
- bounded output, memory, CPU, calls, tokens, and wall time;
- artifact path confinement and regular-file checks, extending current ragopt behavior;
- trace redaction and evaluator-only artifact classes;
- static checks for credential-shaped text, unsafe controls, and evaluation leakage;
- human approval for safety/tool/schema changes;
- non-applying activation plans and atomic rollback pointers;
- complete audit trail of proposer access and artifacts.

16.3 15.3 Mutation policy

The StudySpec should declare mutation classes:

```yaml
mutation_policy:
  allowed:
    - nodes.retrieve.config.*
    - nodes.generate.config.prompt_asset
    - nodes.search_tool.config.description_asset
  denied:
    - evaluators.*
    - datasets.validation.*
    - datasets.audit.*
    - nodes.*.secrets
    - nodes.*.authorization
    - promotion.*
  approval_required:
    - nodes.*.tools
    - nodes.*.output_schema
    - graph.structure
    - script.components
```

The compiler evaluates this policy against the actual patch, not only the proposer’s declaration.

17 16. Concrete changes to ragkit and ragopt

17.1 16.1 ragkit changes

Keep existing APIs stable where possible. Add:

1. **Standard schema registry** for existing `rag` types.
2. **Component descriptors** for chunkers, representation builders, embedders, indexes, retrieval/fusion, rerankers, context policies, generators, and answer validation.
3. **IR adapters** that instantiate existing interfaces from compiled configs.
4. **Identity descriptors** that state exactly which config and input fields determine output.
5. **Cost estimators** for calls, cardinality, memory, and artifact size.
6. **Generic trace payloads** for existing answer stages and index build/open/verify stages.
7. **Provider protocol identities** richer than model name alone.
8. **No semantic scheduler** inside `flow`; keep `flow` as the replayable per-item executor below the plan runtime.

A useful compatibility adapter is:

```
type StepComponent[I, O any] struct {
    DescriptorValue registry.Descriptor
    DecodeInput     func(Envelope) ([]I, error)
    Step            flow.Step[I, O]
    EncodeOutput    func([]flow.Result[O]) (Envelope, error)
}
```

The optimizer sees a node descriptor; runtime code still uses the proven typed step.

17.2 16.2 ragopt changes

Evolve current packages rather than layering another repository on top:

- retain `runstore`, `eval`, `compare`, `gate/policy`, `review`, and reporting semantics;
- generalize `candidate` into `Plan` plus `Patch`, while retaining a legacy exactly-one-asset importer;
- generalize a two-arm run into `Trial` with one or more arms/plans and a persisted block schedule;
- add campaign archive and optimizer state;
- add canonical trace/event types;
- add compiler and registries;
- add fidelity and scheduler policies;
- add statistical comparison and Pareto archive;
- add external executor and artifact-store ports.

The generic `Arm` interface remains valuable as an escape hatch and migration adapter. It should become one `opaque.product-arm` component rather than the only execution model.

17.3 16.3 Compatibility profiles

Provide explicit profiles:

17.3.1 `legacy-paired-v1`

- current suite/candidate/arm interfaces;
- exactly one mutable asset;
- current comparison and gate formats;
- imported into the new event/artifact hierarchy.

17.3.2 `proof-v1`

- compiled plan and one typed target;
- incumbent/challenger paired cells;
- activation/exercise assertions;
- statistical non-inferiority plus existing gate style.

17.3.3 `search-v1`

- campaign, ask/tell optimizer, multiple trials/fidelities;
- Pareto archive;
- bounded multi-target patches.

This prevents a premature rewrite and gives CoinVault a safe migration path.

18 17. Product migration plan

18.1 17.1 Phase 0: freeze vocabulary and evidence schemas

Deliverables:

- RFC for `Study`, `Plan`, `Patch`, `Campaign`, `Trial`, `Cell`, `Attempt`, `Metric`, and `Decision`.
- Versioned JSON schemas and canonicalization rules.
- Artifact and trace envelopes.
- Mapping from current `ragopt` records.
- One generated lockfile format.

Exit criteria:

- current CoinVault and RAG-TTC runs can be imported losslessly into the new read model;
- cell/journal integrity tests pass;
- semantic versus execution identity rules are documented and tested.

Immediate repair: parse or generate CoinVault's runtime contract from the same typed budget object used by execution, and reject any compiled discrepancy.

18.2 17.2 Phase 1: canonical IR, compiler, and registries

Deliverables:

- component/schema registries;
- typed graph IR;
- compiler passes and lockfile;
- Go builder and strict YAML authoring format;
- wrappers for a minimal ragkit path: chunk -> represent -> embed -> index -> retrieve -> generate -> validate.

Exit criteria:

- a fixed RAG plan compiles deterministically;
- plan digest is stable across map order and authoring syntax;
- invalid ports/effects/configs fail before work;
- compiled execution matches the existing direct ragkit fixture.

18.3 17.3 Phase 2: unified runtime, artifacts, traces, and replay

Deliverables:

- local plan executor using `flow` for per-item work;
- content-addressed artifact store;
- event journal and SQLite projection;
- generic node spans and domain payload schemas;
- crash/reconcile/resume protocol;
- persisted randomized paired schedules.

Exit criteria:

- kill-and-resume loses at most declared in-flight work;
- byte and metric replay from immutable artifacts is verified;
- cache reuse follows subgraph identity and cannot cross unsafe candidate boundaries;
- existing ragopt native artifacts remain accessible.

18.4 17.4 Phase 3: migrate both products as plugins and studies

18.4.1 CoinVault

- express the current native system as a SystemSpec or initially as an opaque product component;
- move budgets, models, suites, source identities, and treatment contracts into StudySpec/lockfile generation;
- register mutation targets for result limits, prompts, descriptions, comparison behavior, and reranker config;
- convert trace collector checks into generic assertions plus CoinVault payload schemas;
- remove the central mechanism switch as targets become registered adapters;
- generate run names/descriptions from study metadata rather than the original proof name.

18.4.2 RAG-TTC

- replace the generated `tool-qa.yaml` string with a compiled plan;
- register the orchestration prompt, search description, answer schema, and tool-loop components;
- add treatment activation and broader deterministic metrics;
- express the I5 experiment as a proof StudySpec;
- convert answer-quality and chunk-comparison runners into separate StudySpecs over shared components.

Exit criteria:

- both current proof experiments reproduce their incumbent/candidate projections through the new runtime;
- product-specific code owns semantics only, not run scheduling or generic artifacts;
- no duplicated semantic budget/model configuration remains.

18.5 17.5 Phase 4: search space and multi-fidelity engine

Deliverables:

- domains and MutationCatalog;
- manual, random/Sobol, successive-halving/Hyperband-like optimizers;
- campaign archive and Pareto ranking;
- dependency-aware unique index-build scheduling;
- fidelity promotion policy.

Exit criteria:

- a campaign can screen query parameters and at least one index parameter;

- identical build subgraphs execute once;
- optimizer snapshots and resumes deterministically;
- hard constraints apply at every fidelity.

18.6 17.6 Phase 5: reflective textual proposer

Deliverables:

- feedback packet builder over generic traces/warehouse;
- GEPA-style proposer plugin;
- complete-replacement patch schema;
- leakage and secret scanning;
- textual lesson/ancestry archive;
- feedback-only access enforcement.

Exit criteria:

- a scripted reflector fixture improves a deterministic failure case;
- real proposer patches are reproducible from stored request/response artifacts;
- validation and audit data are inaccessible to the proposer process;
- rejected proposals remain attributable and replayable.

18.7 17.7 Phase 6: statistical/Pareto promotion and production loop

Deliverables:

- hierarchical bootstrap and non-inferiority policies;
- constrained Pareto archive and operating-policy selection;
- blinded pairwise/human review integration;
- audit-set protocol;
- activation plans, canary metrics, and rollback adapters;
- reviewed production-probe intake.

Exit criteria:

- promotion reports include uncertainty, multiple-candidate context, and treatment validity;
- activation remains product/deployment-owned;
- a canary can be rolled back without changing evidence artifacts;
- audit leakage tests and access logs pass.

19 18. Recommended first vertical slice

Build a small end-to-end slice that proves the architecture rather than a large optimizer:

1. One frozen CoinVault or TTC index bundle.
2. A compiled online graph with retrieve, optional rerank, context, generation/tool loop, and contract validation.
3. Three mutable targets:
   - retrieval depth/default results;
   - reranker choice/config;
   - one prompt or tool description text asset.
4. One feedback suite, one validation suite, and one small audit suite.
5. Deterministic metrics plus the existing judge.
6. Generic trace assertions proving each target is active and exercised.
7. Manual and Sobol proposers plus one GEPA-style textual proposer.
8. Paired randomized execution, bootstrap intervals, hard constraints, and a Pareto report.

9. A generated non-applying promotion plan.

This slice touches numeric, categorical, and textual domains; query execution; rich traces; reflective proposal; multi-objective decisions; and product plugins. It avoids index-build complexity until the IR, identities, and evidence model are proven. The next slice adds chunking/representation/index parameters and dependency-aware artifact reuse.

20 19. Acceptance criteria for a proper foundation

The foundation is ready for broader optimization when all of the following are true:

20.1 Correctness and identity

- One StudySpec compiles to one canonical plan/lockfile across machines.
- Every result-affecting field participates in semantic identity.
- Execution-only policy does not poison semantic cache identity.
- Candidate patches are independently diffed and policy-checked.
- Build/query dependency invalidation is deterministic.

20.2 Reproducibility and custody

- Inputs and artifacts are content-addressed and verified.
- Events/cells are append-only and tamper-evident.
- Resume validates exact study, plan, inputs, schedule, and committed evidence.
- Billed usage survives provider errors and uncertain timeouts.
- SQLite/Parquet projections rebuild from authority.

20.3 Extensibility

- A new component is added by descriptor, schema, registration, and conformance tests, not a central switch.
- A new metric or optimizer is added through an interface and registry.
- Product semantics remain in product packages.
- External plugins run with declared capabilities and confined artifacts.

20.4 Optimization quality

- Search supports conditional domains and multiple fidelities.
- The campaign retains ancestry and a feasible Pareto archive.
- Reflection uses rich feedback but cannot see protected splits.
- Paired schedules are randomized/counterbalanced and persisted.
- Promotion reports uncertainty, failures, missingness, and multiple-candidate selection context.

20.5 Governance

- Judge/evaluator and safety policy cannot be mutated by ordinary studies.
- Human review can be structurally blinded.
- Promotion emits a plan; deployment activation is separate.
- Canary monitoring and rollback identities are recorded.
- All mutable surfaces and secret/network effects are explicit.

21 20. Risks and anti-patterns

21.1 20.1 Building a universal workflow engine

The goal is a small typed optimization kernel, not a replacement for Temporal, AWS Batch, River, Kubernetes, or application orchestration. Keep durable job execution behind an adapter.

21.2 20.2 A Turing-complete DSL too early

It would obscure identity, make static analysis weak, and duplicate Go application logic. Prefer declarative graphs plus constrained expressions and explicit script nodes.

21.3 20.3 Central enums and giant switches

They recreate the CoinVault coupling in a new repository. Component and mutation registries must own extensibility.

21.4 20.4 Arbitrary assets without semantic schemas

Files are useful storage, but the framework must know whether a file is a prompt, tool description, index manifest, policy, or script and how it affects identity and security.

21.5 20.5 One scalar reward

It encourages hidden safety, reliability, and cost regressions. Use constraints and a Pareto archive; apply scalar or lexicographic policy only at the final operating decision.

21.6 20.6 Optimizing the judge with the system

This destroys a stable measurement reference and invites reward hacking. Evaluator changes require a new study and recertified incumbent.

21.7 20.7 Leaking validation/audit evidence into reflection

A reflective optimizer can memorize benchmarks rapidly. Enforce access boundaries at the artifact store and process capability level, not only by convention.

21.8 20.8 Unsafe cache sharing

A prompt, model protocol, task prefix, retrieval config, or schema omitted from a cache key can make a candidate appear active while replaying incumbent output. Compiler-generated identity rules and treatment assertions are mandatory.

21.9 20.9 Rebuilding everything for every candidate

This makes indexing optimization unnecessarily expensive. Use content-addressed node outputs and dependency-aware reuse.

21.10 20.10 Treating provider drift as candidate effect

Persist arm order, time blocks, endpoint identity, and environment. Randomize/counterbalance pairs and use repeats. Do not compare runs across incompatible evaluator or provider protocol identities without an explicit bridge study.

21.11 20.11 Best-of-many validation selection

Selecting the highest of many noisy candidates and reporting its ordinary interval is overconfident. Protect audit evidence and account for adaptive search.

21.12 20.12 Dynamic Go plugins as the public contract

Toolchain coupling and ABI fragility conflict with reproducible, independently versioned extensions. Use static registrations or process/WASI boundaries.

22 21. Final recommendation

The current work has already solved many of the hard operational details that generic optimizer projects often ignore: immutable inputs, exact source identity, native artifact custody, conservative budgets, resumability, failure attribution, treatment verification, blinded review, and promotion discipline. The main architectural mistake would be to throw those away in favor of an optimizer-centric library that only suggests prompts and records scalar scores.

The proper foundation is a **compiler-centered experimental systems architecture**:

- `ragkit` supplies deterministic, typed RAG mechanisms;
- a canonical graph IR makes complete systems inspectable and patchable;
- `ragopt` owns immutable studies, campaigns, execution evidence, comparison, and governance;
- plugins supply product semantics and external algorithms;
- search engines operate through ask/tell over typed patches and multi-fidelity results;
- rich traces drive GEPA-style proposals;
- constraints, statistical evidence, Pareto context, and review drive promotion.

The single most important implementation move is to introduce `StudySpec -> canonical Plan/Lockfile` compilation and migrate both existing loops onto it before adding sophisticated search. Once semantic identity, mutation targets, trace assertions, and reusable execution DAGs are explicit, GEPA, BOHB, NSGA-II, MOBO, scripts, and future optimizers become ordinary plugins rather than new application architectures.

23 Appendix A. Suggested Go interfaces

```go
// Canonical authoring and compilation.
type Compiler interface {
    Compile(context.Context, spec.Study) (compiler.Result, error)
}

type CompileResult struct {
    Study           ir.Study
    BasePlans       []ir.Plan
    MutationCatalog ir.MutationCatalog
    Lockfile        ir.Lockfile
    Diagnostics     []Diagnostic
}

// Plan execution.
type Runtime interface {
    ExecuteNode(context.Context, ir.NodeExecution,
        trace.Emitter) (artifact.Ref, error)
}

// Durable control plane.
```

```go
type CampaignStore interface {
    Append(context.Context, study.Event) error
    LoadCampaign(context.Context, string) (study.CampaignView, error)
    Claim(context.Context, study.WorkItem) (study.Lease, error)
    Commit(context.Context, study.Lease, study.Commit) error
}

// Search.
type Optimizer interface {
    Initialize(context.Context, search.StudyView) error
    Ask(context.Context, search.AskRequest) ([]ir.Patch, error)
    Tell(context.Context, []search.TrialObservation) error
    Snapshot(context.Context) (artifact.Ref, error)
}

// Decision.
type PromotionPolicy interface {
    Evaluate(context.Context, gate.Evidence) (gate.Decision, error)
}
```

24 Appendix B. Suggested event record

```json
{
  "api_version": "ragopt-event/v1alpha1",
  "sequence": 184,
  "event_id": "evt-...",
  "previous_digest": "sha256:...",
  "digest": "sha256:...",
  "at": "2026-08-12T15:01:02Z",
  "kind": "metric.recorded",
  "study_id": "study-...",
  "campaign_id": "campaign-...",
  "trial_id": "trial-...",
  "cell_id": "cell-...",
  "attempt_id": "attempt-2",
  "node_id": "generate",
  "payload": {
    "schema": "rag.metric-observation/v1",
    "artifact": {
      "digest": "sha256:...",
      "size_bytes": 842,
      "media_type": "application/json"
    }
  }
}
```

25 Appendix C. Source map for the code findings

25.1 CoinVault/GEC

- `cmd/coinvault/cmds/knowledge_ragopt.go:50-53` - locked budget constants.
- `cmd/coinvault/cmds/knowledge_ragopt.go:145-277` - full command lifecycle, preflight, resume, paired run, terminal gate, and reporting.
- `cmd/coinvault/cmds/knowledge_ragopt.go:304-305` - command rejects budget changes from code constants.

- `cmd/coinvault/cmds/knowledge_ragopt.go:434-543` - product-specific treatment mechanism selection and runtime configuration.
- `cmd/coinvault/cmds/knowledge_ragopt.go:545-605` and following - native execution, timeout ownership, trace and failure handling.
- `cmd/coinvault/cmds/knowledge_ragopt_trace.go:127-300` - provider, tool, evidence, identity, result-limit, reranking, and terminal trace collection.
- `cmd/coinvault/cmds/knowledge_ragopt_{case,contract,gate,reranker,suite_lock,treatment}.go` - adjacent product contracts and gates.
- `configs/ragopt/default-results-8-v7/shared/runtime-contract.yaml` - locked runtime declaration with budget values that differ from command constants.
- `configs/ragopt/default-results-8-v7/{parent,candidate}/snapshot.yaml` - runtime contract included as a locked asset.

25.2 ragkit

- `README.md:1-61` - contracts-first package map, invariants, dependency boundaries, and provider ownership.
- `rag/types.go:3-64` - document, exact chunk, representation, vector, query, and judgment types.
- `rag/components.go:8-88` - narrow chunker, generator, embedder, search/index, and reranker interfaces.
- `flow/doc.go:1-17` - generic bounded replay layer; explicitly not a durable workflow/DAG system.
- `flow/step.go:13-71` - semantic identity, execution policy, typed work, meters, and composition metadata.
- `flow/policy.go:9-58` - workers, fail-closed admission, retries, quarantine, and skip policy.
- `rag/indexbundle/types.go:18-75` - immutable bundle identities and build inputs.
- `rag/answering/service.go:16-39` - semantic RAG pipeline ownership and prompt/contract identity.
- `rag/answering/service.go:54-139` - closed retrieval strategy validation.
- `rag/answering/types.go:59-204` - inspectable retrieval/answer results and stage observations.

25.3 RAG-TTC

- `cmd/rag-ttc/cmds/tooleval/ragopt.go:85-202` - candidate/suite/environment setup and paired ragopt run.
- `cmd/rag-ttc/cmds/tooleval/ragopt.go:302-357` - native cell execution, session projection, judging, and artifact creation.
- `cmd/rag-ttc/cmds/tooleval/ragopt.go:360-375` - narrow generic metric/cost projection.
- `cmd/rag-ttc/cmds/tooleval/ragopt.go:415-426` - hard-coded generated tool runtime YAML.
- `cmd/rag-ttc/cmds/experiments/answerquality/runner.go` - broader answer-quality experiment orchestration.
- `cmd/rag-ttc/cmds/experiments/chunkcompare/run.go` - separate chunking experiment orchestration.
- `ttmp/2026/08/02/RAG-TTC-GEPA-OPT-001.../design-doc/01-intern-guide-to-a-pragmatic-gepa-inspired-self-optimization-loop.md` - reflection, warehouse, splits, candidate, evaluation, and promotion design.
- `ttmp/2026/08/02/RAG-TTC-TOOLLOOP-001.../design-doc/02-beyond-benchmark-saturation-a-pragmatic-ttc-rag-optimization-roadmap.md` - frozen regression, diagnostic, and production-probe layers.
- `ttmp/2026/08/08/RAG-TTC-CLEANUP-001.../design-doc/01-intern-guide-to-post-cutover-cleanup-ownership-boundaries-and-staged-refactoring.md` - ragkit/ragopt/product ownership boundaries.

25.4 ragopt

CoinVault references commit `4d410c57e242`; RAG-TTC references `v0.0.1` at `0e9c584fee2d`, 53 commits ahead of the CoinVault revision.

- `pkg/candidate/types.go` and `candidate.go` - strict snapshots, hypotheses, evidence, and exactly-one-mutable-asset verification.
- `pkg/eval/types.go` and `runner.go` - opaque product cases, generic outcomes, product `Arm`, paired schedule, native artifact confinement, and resume.
- `pkg/runstore/types.go` - append-only active run and immutable terminal run contract.
- `pkg/compare/types.go` - paired deltas and group/metric aggregates.
- `pkg/gate/policy.go` in the earlier revision, moved to `pkg/policy` by v0.0.1 - hard gates, target, regressions, and tie-breakers.
- `pkg/eval/cell_chain.go` in v0.0.1 - chained cell digests.
- `pkg/eval/evidence_guard.go` in v0.0.1 - trust-root and journal mutation guard around arm execution.
- `pkg/review/review.go` in v0.0.1 - deterministic structurally blinded review queue and separate key.
- `ttmp/.../design-doc/02-production-index-build-scheduling-resumability-and-ragopt-integration.md` - product/control-plane/external-scheduler responsibility split.

26 Appendix D. Design influences and references

1. Lakshya A. Agrawal et al. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning, 2025. Relevant ideas: trajectory-based natural-language reflection, complete prompt mutation, and Pareto retention.
2. Haiqiang Zhang et al. RAG-Stack: Co-Optimizing RAG Serving Performance and Quality, 2026. Relevant ideas: RAG intermediate representation, performance model, plan exploration, and quality/performance Pareto frontiers.
3. Wenqi Jiang. RAG-Stack: Co-Optimizing RAG Quality and Performance From the Vector Database Perspective, 2025. Earlier three-pillar blueprint for RAG-IR, cost modeling, and plan exploration.
4. Yiqun Chen et al. Improving Retrieval-Augmented Generation through Multi-Agent Reinforcement Learning, 2025. Relevant warning and opportunity: independently optimized RAG modules can be misaligned with end-to-end answer objectives.
5. Kalyanmoy Deb et al. "A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II." *IEEE Transactions on Evolutionary Computation* 6(2), 2002. DOI: 10.1109/4235.996017.
6. Lisha Li et al. Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization. *JMLR* 18(185), 2018.
7. Stefan Falkner, Aaron Klein, and Frank Hutter. BOHB: Robust and Efficient Hyperparameter Optimization at Scale. *ICML/PMLR* 80, 2018.
8. James S. Bergstra et al. Algorithms for Hyper-Parameter Optimization. *NeurIPS* 24, 2011. Relevant idea: model conditional/hierarchical spaces rather than treating every parameter as always active.
9. Samuel Daulton, Maximilian Balandat, and Eytan Bakshy. Parallel Bayesian Optimization of Multiple Noisy Objectives with Expected Hypervolume Improvement, 2021. Relevant idea: noisy multi-objective batch selection.